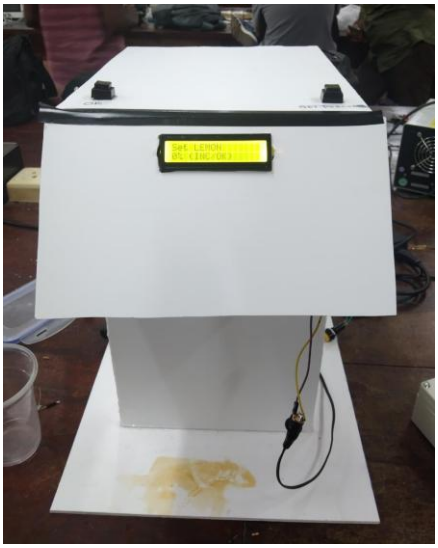
DEPARTMENT OF PHYSICS

UNIVERSITY OF COLOMBO

PH 3032 - Embedded Systems Laboratory

Automatic Lemon Juice Vending Machine



Name :- S.K.S.Lasal

Index :- s16818

Reg.No :- 2022s19438

Table of content

1. Abstract 3
2. Introduction 4
3. Methodology 5
   3.1 Components 5
   3.2 Schematic Diagram 9
   3.3 System Initialization 10
   3.4 PCB design 11
   3.5 Enclosure Design 12
   3.6 User Interface 13
   3.7 Testing and Calibration 14
4. Results and Analysis 15
5. Discussion 17
6. Conclusion 18
References 19
Appendix 20
Main code 20
Header file code for i2C communication 25

1. Abstract

This project presents the design and implementation of a microcontroller-based lemonade mixing machine, capable of dispensing three ingredients lemon juice, sugar syrup, and water in user-defined proportions. The system is built around an ATmega328P microcontroller and operates in Manual Mode, where users can select the percentage of lemon and sugar syrup using push buttons. The remaining percentage is automatically assigned to water, ensuring a total of 100%. The dispensing process is executed sequentially, with each pump powered individually under microcontroller control, guaranteeing that all motors remain off until the mixing cycle begins. An LCD display with I^2C interface provides real-time instructions, percentage selection, and status updates during operation, offering a simple and user-friendly interface.

The use of peristaltic pumps ensures accurate dispensing and reduces contamination, while the microcontroller-based timing mechanism maps the selected percentages to precise pump activation durations. This design achieves consistent and repeatable results, making it suitable for small-scale automated beverage preparation. The system highlights the integration of embedded control, user interaction, and automation in a compact design. Overall, this lemonade machine demonstrates a reliable and customizable approach to beverage mixing, with potential for expansion into multi-flavor automated drink systems for domestic and commercial applications.

2. Introduction

The demand for customizable beverages has increased significantly in recent years, both in domestic and commercial environments. Consumers increasingly prefer drinks tailored to their personal taste, such as specific levels of sweetness or acidity. Traditional manual preparation, however, can be inconsistent and time-consuming. To address this need, automation offers a practical solution for ensuring accuracy, repeatability, and ease of operation.

This project introduces a microcontroller-based lemonade mixing machine that allows users to create a refreshing beverage by selecting the proportions of lemon juice, sugar syrup, and water. The machine is built using an ATmega328P microcontroller, push buttons for user input, and an LCD display with I^2C interface for guiding the user throughout the process. The system is designed to be simple and interactive: the user sets the percentages of lemon and sugar syrup, while the remaining percentage is automatically assigned to water, ensuring the total always equals 100%.

The machine operates by activating peristaltic pumps connected to each ingredient container. These pumps are controlled by the microcontroller and are powered sequentially to dispense the exact ratio selected by the user. Once the process is complete, the system automatically turns off all pumps and displays a completion message. This ensures both energy efficiency and safety, while also minimizing waste.

The project demonstrates how embedded systems can be applied to automate beverage preparation. By combining user-friendly design with precise control, the lemonade machine offers a reliable and efficient solution for customizable drink mixing. It also serves as a proof of concept for future beverage automation systems that could include multiple flavors, automated cleaning cycles, or integration into smart kitchen devices.

3. Methodology

3.1 Components

- ATmega328P Microcontroller
- 230 V to 12 V/ 1 A power pack
- 12 V to 5 V step-down buck converter
- 12 V peristaltic pump motors
- 5V relays
- LCD display
- i2C communication module for LCD display
- 5V operable push button
- transparent tubes
- Circuit Wires

3.1.1 Atmega328p Microcontroller

The ATmega328P is an 8-bit AVR microcontroller widely used in embedded system applications due to its simplicity, reliability, and low power consumption. It operates at clock speeds up to 16 MHz and features 32 KB of Flash memory**,** 2 KB of SRAM, and 1 KB of EEPROM, making it suitable for small to medium-scale projects.

It includes several built-in peripherals such as digital I/O pins, analog-to-digital converters (ADC), timers, PWM outputs, UART, SPI, and I^2C communication interfaces. These features allow easy integration with external modules like LCD displays, sensors, and motor drivers.

In this project, the ATmega328P acts as the central controller, handling button inputs, LCD communication, and motor control. It ensures that the pumps are activated for precise durations based on the user-selected percentages, providing accurate beverage mixing.

Figure 1 atmega328p microcontroller

3.1.2 12 V to 5 V step-down buck converter

A 12 V to 5 V step-down buck converter is used in this project to provide a stable 5 V supply for the ATmega328P microcontroller and other control components. It efficiently reduces the 12 V input, which powers the pumps, down to 5 V for the logic circuit, ensuring reliable operation with minimal power loss compared to linear regulators.

Figure 2 12V to 5V buck converter

3.1.3 12 V peristaltic pump motors

The 12 V peristaltic pump motors are used to accurately dispense different juices in the machine. They work by compressing and releasing flexible tubing, which moves the liquid forward without the liquid ever touching the pump mechanism. This design prevents contamination, ensures hygiene, and provides precise control of juice flow for consistent mixing.

Figure 3 12V peristatic pump motors

3.1.4 Relays

Relays are used in this project to control the 12 V peristaltic pump motors using the 5 V signals from the ATmega328P. They act as electrically operated switches, isolating the low-voltage control circuit from the high-power motor circuit, ensuring safe and reliable operation of the pumps.

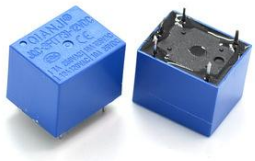


Figure 4 relay

3.1.5 LCD display

The LCD display is used to show system information, such as mode selection, juice percentages, and status messages, providing a clear interface for the user.

Figure 5 LCD display

3.1.6 i2C communication module

The I2C communication module for the LCD reduces the number of microcontroller pins required by allowing the ATmega328P to communicate with the LCD using only two lines (SDA and SCL), simplifying wiring and saving resources.

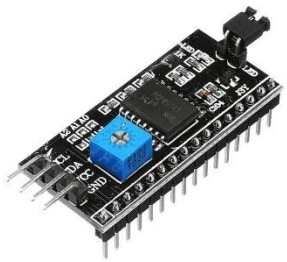


Figure 6 i2c communication module

3.1.7 5V operable push button

The 5 V operable push buttons are used for user input, enabling mode selection, percentage adjustments, and starting or stopping the juice dispensing process. They provide a simple and reliable way for users to interact with the machine.

Figure 7 push button

3.2 Schematic Diagram

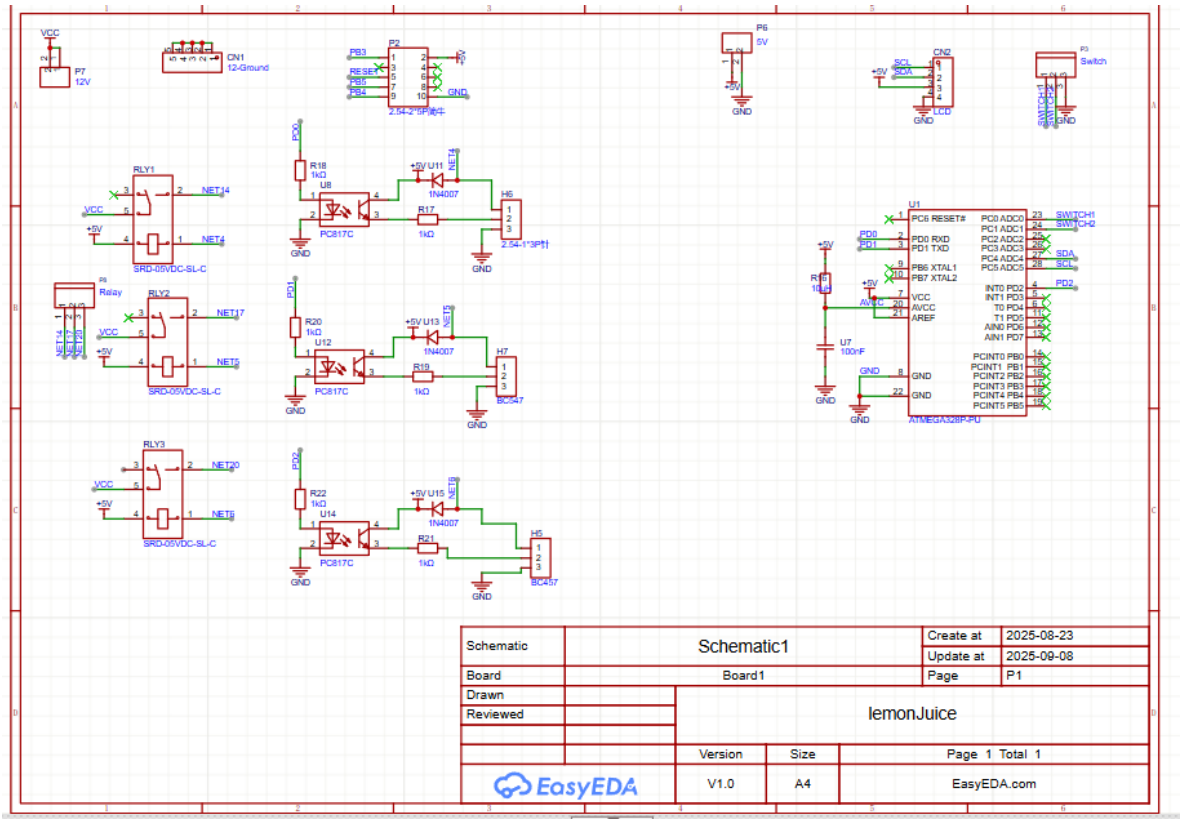


Figure 8 schematic diagram for designing the PCB design of the project

This circuit diagram shows the setup for a lemon mixing machine controlled by an ATmega328P microcontroller. The ATmega328P manages relays to control pumps, switch inputs for mode selection, and an LCD display via I2C for user interface. This configuration enables precise control of juice flow and selection, making it suitable for customized beverage mixing.

3.3 System Initialization

3.3.1 Programming Environment and Tools

The software for the ATmega328P microcontroller was developed using the Ubuntu terminal, with AVR-GCC as the compiler and AVRDUDE for uploading the firmware. A custom I2C header file was used to simplify communication with the LCD display, reducing wiring and code complexity. This setup provided an efficient and reliable development environment.

3.3.2 Firmware for ATmega328P Microcontroller

The firmware was written to control the peristaltic pump motors, relays, push buttons, and LCD display, ensuring that the juice mixing process happens step by step in a controlled sequence. The program is organized as follows:

1. Initialization: All I/O pins, relays, LCD, and push buttons are configured at startup.

2. Main Control Loop: The system waits for the user command through the push buttons.

3. Sequential Dispensing: When the user starts the process, the firmware first powers the lemon motor for the set time, then turns it off. Next, the sugar syrup motor is powered and turned off after dispensing. Finally, the water motor is activated to complete the lemonade. After the sequence, all motors remain off until a new command is given.

4. Relay and Motor Control: Relays are used to switch the 12 V motors using safe 5 V signals from the microcontroller.

5. LCD Display Updates: The LCD provides real-time feedback, such as “Dispensing Lemon,” “Adding Sugar,” “Adding Water,” and “Drink Ready.”

6. Safety Feature: Motors are always off by default, and they only activate when commanded, preventing accidental dispensing.

3.4 PCB design

The PCB design for the juice mixing machine was developed using EasyEDA, ensuring a compact and organized layout for all components. The ATmega328P microcontroller was placed at the core of the board to manage all control signals.

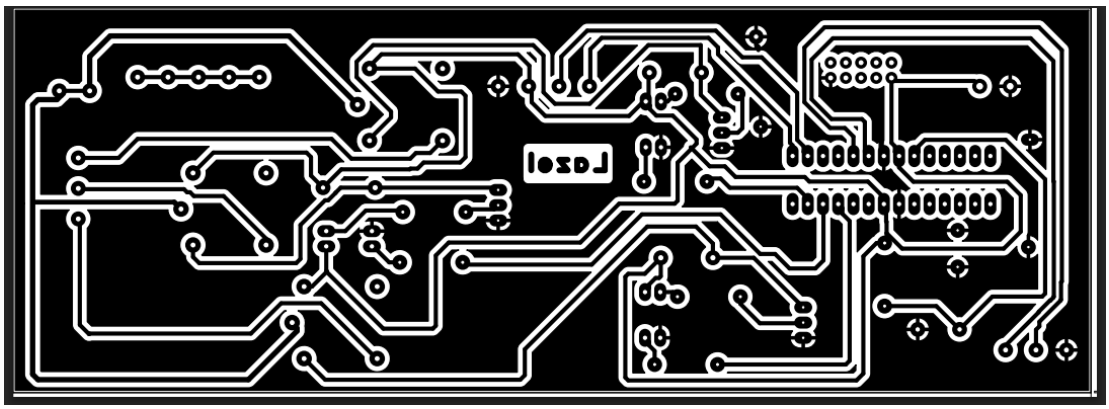


Figure 9 PCB design

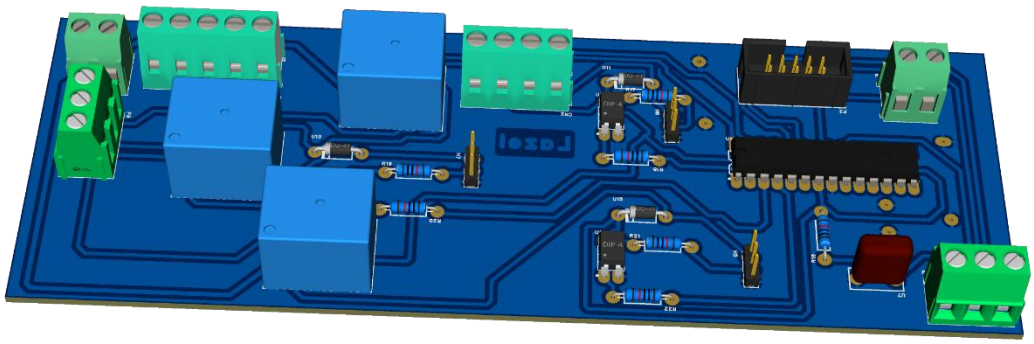


Figure 10 3D design of PCB

3.5 Enclosure Design

The enclosure design of the juice mixing machine was made using sunboard (plastic board), chosen for its lightweight, low cost, and easy workability. The sunboard sheets were precisely cut to the required dimensions using a paper cutter, allowing smooth edges and proper fitting. The cut pieces were then assembled and joined using hot glue, which provided a firm hold and ensured the structure was stable. This enclosure not only protected the internal electronic components such as the PCB, relays, and wiring but also gave the machine a neat and professional appearance. The use of sunboard made the design simple to fabricate, while keeping the enclosure durable and easy to modify if required.

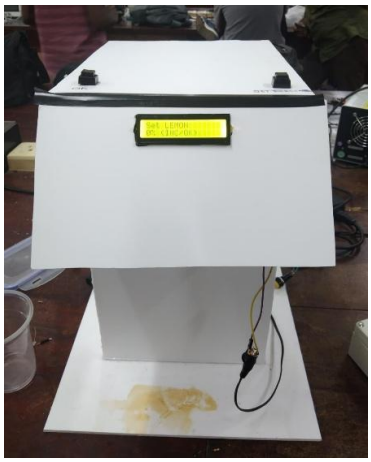


Figure 11 Automatic Lemon Juice Vending Machine

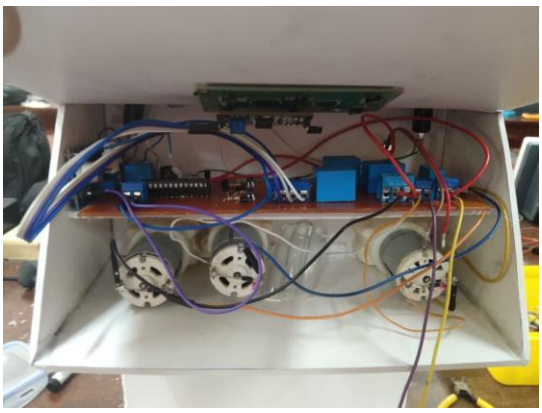


Figure 12 PCB and Pump motors

3.6 User Interface

The user interface of the juice mixing machine is provided through the LCD display, which shows clear instructions and status messages. It guides the user step by step, starting with the selection of juice percentages for lemon and sugar, automatically calculating the remaining water percentage. During operation, the LCD shows which juice is being dispensed and its percentage. Finally, it displays a completion message like *"Enjoy Your Drink"*. This simple interface makes the system easy to operate, even for first-time users.

Figure 16 step 01

Figure 15 set lemon percentage

Figure 14 set sugar percentage

Figure 13 display all percentage

Figure 18 dispensing lemon

Figure 17 dispensing sugar

Figure 20 dispensing water

Figure 19 finally ready drink

3.7 Testing and Calibration

3.7.1 Calibration of Dispensing Volumes

For testing and calibration, the peristaltic pumps were timed to measure the volume of liquid dispensed. It was observed that running a pump for 6500ms fills approximately 250ml of liquid.

0% = 0 ml

20% = 50 ml

40% = 100 ml

60% = 150 ml

80% = 200 ml

100% = 250 ml

3.7.2 Testing Dispensing Times for Calibration

For each motor, the following procedure was conducted to determine the time delay required to achieve the specified volume levels:

1. Setting Up the Measurement:

The cup was positioned, and each motor was turned on individually to dispense juice to the designated volume marks (0 ml, 50 ml, 100 ml, 150 ml, 200 ml, and 250 ml).

2. Timing the Dispensing:

The time taken by each motor to fill the cup to each boundary mark was recorded. This process was repeated multiple times for each motor, and an average time delay was calculated for each volume increment (20%, 40%, 60%, 80%, and 100%) to improve accuracy.

2. Delay Time Calculation:

Based on the average time delay obtained for each volume increment, a delay time was assigned to each percentage setting. This ensured that the motor would run for the correct duration to dispense the desired amount.

4. Results and Analysis

To achieve accurate dispensing, each motor was calibrated to dispense specific volumes based on user-selected percentages. The calibration was carried out using a cup marked with boundaries corresponding to different volume levels, from 0 ml (0%) up to 250 ml (100%).The time delay measurements for each motor were recorded and averaged. Minor variations across the motors were observed, attributed to individual motor characteristics.

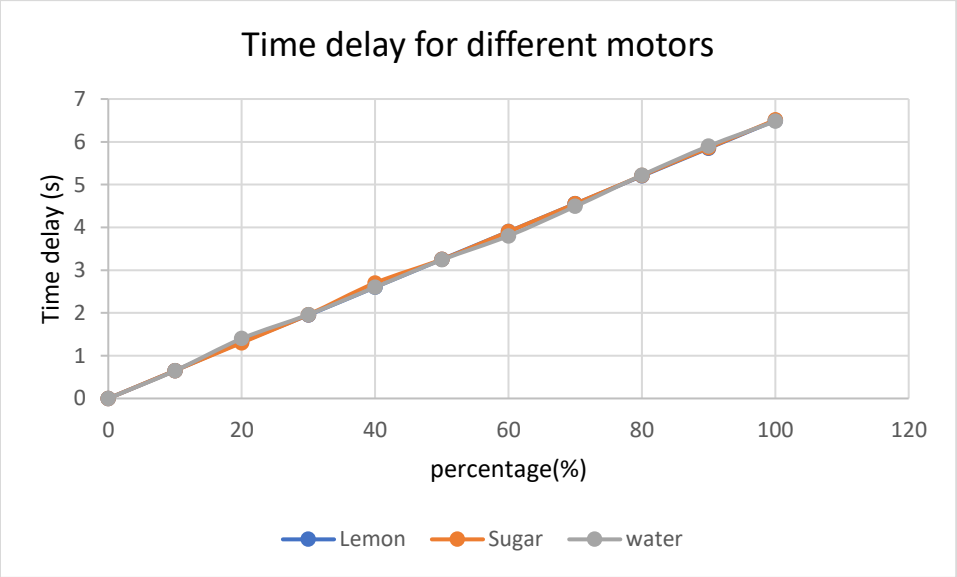


Figure 21 time delay vs dispending percentage for three motors

Obtained corresponding average time delays for each quantity as follows

- 0% (0 ml): 0 seconds
- 10% (25 ml): 0.65 seconds
- 20% (50 ml): 1.32 seconds
- 30% (75 ml): 1.95 seconds
- 40% (100 ml): 2.6 seconds
- 50% (125 ml): 3.25 seconds
- 60% (150 ml): 3.9 seconds
- 70% (175 ml): 4.55 seconds
- 80% (200 ml): 5.2 seconds
- 90% (225 ml): 5.85 seconds
- 100% (250 ml): 6.5 seconds

5. Discussion

The developed juice mixing machine successfully demonstrated the ability to prepare customized lemonade by accurately controlling the dispensing of lemon juice, sugar syrup, and water. The use of the ATmega328P microcontroller allowed for precise timing control of the peristaltic pumps, ensuring that the selected percentages corresponded closely to the actual dispensed volumes. Through calibration, it was established that 6500 ms of pump operation delivers 250 ml, which provided a reliable basis for calculating the dispensing time for different percentages.

The LCD-based user interface made the system simple to operate, guiding the user through the selection process and showing real-time updates during dispensing. By implementing a sequential motor control strategy, the design ensured that only one motor operated at a time, reducing power consumption and avoiding cross-contamination between liquids.

While the system performed well in tests, some limitations were observed. Small variations in pump speed and liquid viscosity could slightly affect the final volume accuracy. Additionally, the machine is currently limited to three ingredients, which restricts its flexibility compared to commercial juice dispensers. However, the modular firmware and PCB design allow for easy expansion if more pumps or ingredients are required.

Overall, the project highlights the effectiveness of combining embedded control, simple user interfaces, and peristaltic pumps for beverage automation. It provides a cost-effective prototype suitable for small-scale use, and with further improvements in enclosure design and pump calibration, it has potential applications in cafes, restaurants, or household settings.

6. Conclusion

The juice mixing machine project successfully achieved its goal of creating a simple, low-cost, and customizable beverage dispenser. Using the ATmega328P microcontroller, the system was able to control peristaltic pumps with precise timing, ensuring accurate proportions of lemon juice, sugar syrup, and water. The LCD-based user interface provided clear guidance, making the machine easy to operate even for new users. Testing and calibration confirmed reliable performance, with dispensing times mapped to liquid volumes for consistent results. Although some minor inaccuracies may occur due to pump variations and liquid properties, the system proved to be effective for small-scale applications. Overall, the project demonstrates the practical use of embedded systems in automating beverage preparation and serves as a strong foundation for future improvements, such as supporting more ingredients or enhancing accuracy with flow sensors.

References

- Zylauv (2019) DIY Arduino cocktail machine, Hackster.io. Available at: https://www.hackster.io/by-zylauv/diy-arduino-cocktail-machine-0eee04
- I2C LCD on Arduino - easily setup and control with an Uno Make. Available at: https://www.make-it.ca/i2c-lcd-display-on-arduino/
- Avrdudes (no date) Avrdudes/Avrdude: AVRDUDE is a utility to program AVR microcontrollers, GitHub. Available at: https://github.com/avrdudes/avrdude
- 4-channel Relay Driver Circuit and PCB Design (no date) Circuit Digest - Electronics Engineering News, Latest Products, Articles and Projects. Available at: https://circuitdigest.com/electronic-circuits/relay-driver-module-circuit-pcb

Appendix

Main code

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <stdio.h>
#include "LCD.h"

/* ---------- Configuration ---------- */
#define STEP 10             // % increment per button press
#define INC_BUTTON_PIN PC0  // Increment button
#define OK_BUTTON_PIN  PC1  // OK / Confirm button

/* Motors: PD0 = LEMON, PD1 = SUGAR, PD2 = WATER (active HIGH = ON) */

/* ---------- Function Prototypes ---------- */
void setup(void);
uint8_t isIncPressed(void);
uint8_t isOkPressed(void);
void waitForButtonRelease(uint8_t pin);
void selectPercentage(const char *name, uint8_t *value);
void startDispenseSequence(uint8_t lemon, uint8_t sugar, uint8_t water);
void turnOnMotor(uint8_t motor, uint8_t percentage);
void allMotorsOff(void);
uint16_t getDelayForPercentage(uint8_t percentage);
void displayExceed100(void);
void displayEnjoyDrink(void);

/* ---------- Main ---------- */
int main(void) {
    setup();
    initialize(); // LCD initialization

    while (1) {
        uint8_t lemon = 0;
        uint8_t sugar = 0;
        uint8_t water = 0;

        // Intro message
        lcd_clear();
        lcd_setCursor(0, 0);
        lcd_print("Manual Lemonade");
```

```c
        lcd_setCursor(0, 1);
        lcd_print("INC=PC0 OK=PC1");
        _delay_ms(2000);

        // Select lemon %
        selectPercentage("LEMON", &lemon);

        // Select sugar %
        selectPercentage("SUGAR", &sugar);

        // Validate and compute water %
        if ((uint16_t)lemon + (uint16_t)sugar > 100) {
            displayExceed100();
            continue; // restart selection
        }
        water = 100 - (lemon + sugar);

        // Show summary
        char buf[17];
        lcd_clear();
        lcd_setCursor(0, 0);
        snprintf(buf, sizeof(buf), "L:%d%% S:%d%%", lemon, sugar);
        lcd_print(buf);
        lcd_setCursor(0, 1);
        snprintf(buf, sizeof(buf), "W:%d%% Press OK", water);
        lcd_print(buf);

        // Wait for OK
        while (!isOkPressed()) {
            _delay_ms(50);
        }
        waitForButtonRelease(OK_BUTTON_PIN);

        // Start dispensing sequence
        startDispenseSequence(lemon, sugar, water);

        // Done
        displayEnjoyDrink();
        _delay_ms(1000);
    }
}

/* ---------- Setup & Buttons ---------- */
void setup(void) {
    // Buttons as input with pull-up
```

```
    DDRC &= ~((1 << INC_BUTTON_PIN) | (1 << OK_BUTTON_PIN));
    PORTC |= (1 << INC_BUTTON_PIN) | (1 << OK_BUTTON_PIN);

    // Motors PD0..PD2 as output
    DDRD |= (1 << PD0) | (1 << PD1) | (1 << PD2);
    allMotorsOff(); // ensure OFF at startup
}

uint8_t isIncPressed(void) {
    return !(PINC & (1 << INC_BUTTON_PIN));
}
uint8_t isOkPressed(void) {
    return !(PINC & (1 << OK_BUTTON_PIN));
}
void waitForButtonRelease(uint8_t pin) {
    while (!(PINC & (1 << pin))) _delay_ms(10);
    _delay_ms(30);
}

/* ---------- UI Helpers ---------- */
void selectPercentage(const char *name, uint8_t *value) {
    char buf[17];
    *value = 0;

    lcd_clear();
    lcd_setCursor(0, 0);
    lcd_print("Set ");
    lcd_print(name);
    lcd_setCursor(0, 1);
    snprintf(buf, sizeof(buf), "%d%% (INC/OK)", *value);
    lcd_print(buf);

    while (1) {
        if (isIncPressed()) {
            _delay_ms(50);
            if (isIncPressed()) {
                if (*value + STEP > 100) {
                    *value = 0; // wrap around
                } else {
                    *value += STEP;
                }
                lcd_setCursor(0, 1);
                snprintf(buf, sizeof(buf), "%d%% (INC/OK)  ", *value);
                lcd_print(buf);
                waitForButtonRelease(INC_BUTTON_PIN);
```

```
            }
        }
        if (isOkPressed()) {
            _delay_ms(50);
            if (isOkPressed()) {
                waitForButtonRelease(OK_BUTTON_PIN);
                break;
            }
        }
        _delay_ms(30);
    }
}

void displayExceed100(void) {
    lcd_clear();
    lcd_setCursor(0, 0);
    lcd_print("Total > 100%");
    lcd_setCursor(0, 1);
    lcd_print("Retry selection");
    _delay_ms(2000);
}

void displayEnjoyDrink(void) {
    lcd_clear();
    lcd_setCursor(0, 0);
    lcd_print("Enjoy");
    lcd_setCursor(0, 1);
    lcd_print("Your drink :)");
    _delay_ms(3000);
}

/* ---------- Dispensing ---------- */
void startDispenseSequence(uint8_t lemon, uint8_t sugar, uint8_t water) {
    char buf[17];

    // Lemon
    if (lemon > 0) {
        lcd_clear();
        lcd_setCursor(0, 0);
        lcd_print("Dispensing Lemon");
        lcd_setCursor(0, 1);
        snprintf(buf, sizeof(buf), "%d%%", lemon);
        lcd_print(buf);
        turnOnMotor(PD0, lemon);
    }
```

```
    // Sugar
    if (sugar > 0) {
        lcd_clear();
        lcd_setCursor(0, 0);
        lcd_print("Dispensing Sugar");
        lcd_setCursor(0, 1);
        snprintf(buf, sizeof(buf), "%d%%", sugar);
        lcd_print(buf);
        turnOnMotor(PD1, sugar);
    }

    // Water
    if (water > 0) {
        lcd_clear();
        lcd_setCursor(0, 0);
        lcd_print("Dispensing Water");
        lcd_setCursor(0, 1);
        snprintf(buf, sizeof(buf), "%d%%", water);
        lcd_print(buf);
        turnOnMotor(PD2, water);
    }

    allMotorsOff(); // ensure all OFF at the end
}

/* ---------- Motor control (Active HIGH) ---------- */
void turnOnMotor(uint8_t motor, uint8_t percentage) {
    if (percentage == 0) return;
    uint16_t delay_ms = getDelayForPercentage(percentage);

    PORTD |= (1 << motor);  // ON (active HIGH)
    while (delay_ms--) {
        _delay_ms(1);
    }
    PORTD &= ~(1 << motor); // OFF
}

void allMotorsOff(void) {
    PORTD &= ~((1 << PD0) | (1 << PD1) | (1 << PD2)); // all OFF (LOW)
}

uint16_t getDelayForPercentage(uint8_t percentage) {
    // Linear mapping: 100% -> ~6500 ms
    return (uint16_t)percentage * 65;
```

```
}
```

Header file code for i2C communication

```
#ifndef LCD_H
#define LCD_H

#include <avr/io.h>
#include <util/delay.h>
#include <compat/twi.h>

// LCD I2C address (usually 0x27 or 0x3F depending on your module)
#define LCD_I2C_ADDRESS 0x3F

// LCD Control bits
#define LCD_BACKLIGHT 0x08  // On
#define LCD_ENABLE    0x04  // Enable bit
#define LCD_RW        0x02  // Read/Write bit
#define LCD_RS        0x01  // Register select bit

// I2C initialization
void i2c_init(void) {
    TWSR = 0x00; // Set prescaler bits to 0
    TWBR = 0x0C; // SCL frequency is 400kHz with 16MHz clock
    TWCR = (1 << TWEN); // Enable TWI (I2C)
}

// Send START condition on I2C
void i2c_start(void) {
    TWCR = (1 << TWSTA) | (1 << TWEN) | (1 << TWINT); // Send START condition
    while (!(TWCR & (1 << TWINT))); // Wait for START to be transmitted
}

// Send STOP condition on I2C
void i2c_stop(void) {
    TWCR = (1 << TWSTO) | (1 << TWEN) | (1 << TWINT); // Send STOP condition
}

// Write data to I2C
void i2c_write(uint8_t data) {
```

```
    TWDR = data; // Load data to data register
    TWCR = (1 << TWEN) | (1 << TWINT); // Start transmission of data
    while (!(TWCR & (1 << TWINT))); // Wait for data to be transmitted
}

// Enable data transmission to LCD
void lcd_enable(uint8_t data) {
    i2c_start();
    i2c_write(LCD_I2C_ADDRESS << 1); // Send address with write bit
    i2c_write(data | LCD_ENABLE);    // Send data with enable bit set
    _delay_us(1);
    i2c_write(data & ~LCD_ENABLE);   // Clear enable bit
    _delay_us(50);
    i2c_stop();
}

// Send data/command to the LCD
void lcd_send(uint8_t data, uint8_t mode) {
    uint8_t highNibble = (data & 0xF0) | mode | LCD_BACKLIGHT;
    uint8_t lowNibble = ((data << 4) & 0xF0) | mode | LCD_BACKLIGHT;

    lcd_enable(highNibble);
    lcd_enable(lowNibble);
}

// Send command to the LCD
void lcd_command(uint8_t cmd) {
    lcd_send(cmd, 0);
}

// Print string on the LCD
void lcd_print(char *str) {
    while (*str) {
        lcd_send(*str, LCD_RS);
        str++;
    }
}

// Clear the LCD screen
void lcd_clear(void) {
    lcd_command(0x01); // Clear display command
    _delay_ms(2);      // Wait for the command to execute
}

// Set cursor position on the LCD
```

```c
void lcd_setCursor(uint8_t col, uint8_t row) {
    uint8_t row_offsets[] = {0x00, 0x40, 0x14, 0x54};
    lcd_command(0x80 | (col + row_offsets[row]));  // Set DDRAM address
}

// Initialize the LCD
void initialize(void) {
    _delay_ms(50);        // Wait for LCD to power up
    lcd_command(0x02);    // Initialize in 4-bit mode
    lcd_command(0x28);    // 2 line, 5x7 matrix
    lcd_command(0x0C);    // Display on, cursor off
    lcd_command(0x06);    // Increment cursor
    lcd_clear();          // Clear display
}

#endif // LCD_H
```